In [ ]:
```
from google.colab import files
files.upload()
```

In [ ]:
```
import pandas as pd

try:
    df = pd.read_csv('/content/Global_Superstore2 .csv', encoding='latin1')
except UnicodeDecodeError:
    df = pd.read_csv('/content/Global_Superstore2 .csv', encoding='ISO-8859-1')

df.head()
data = df.copy()
data
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	
0	32298	CA-2012-124891	31-07-2012	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New
1	26341	IN-2013-77878	05-02-2013	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollon
2	25330	IN-2013-71249	17-10-2013	18-10-2013	First Class	CR-12730	Craig Reiter	Consumer	Bris
3	13524	ES-2013-1579342	28-01-2013	30-01-2013	First Class	KM-16375	Katherine Murray	Home Office	
4	47221	SG-2013-4320	05-11-2013	06-11-2013	Same Day	RH-9495	Rick Hansen	Consumer	
...	...	...	...	...	...	...	...	...	
51285	29002	IN-2014-62366	19-06-2014	19-06-2014	Same Day	KE-16420	Katrina Edelman	Corporate	
51286	35398	US-2014-102288	20-06-2014	24-06-2014	Standard Class	ZC-21910	Zuschuss Carroll	Consumer	Hou
51287	40470	US-2013-155768	02-12-2013	02-12-2013	Same Day	LB-16795	Laurel Beltran	Home Office	Ox
51288	9596	MX-2012-140767	18-02-2012	22-02-2012	Standard Class	RB-19795	Ross Baird	Home Office	Val
51289	6147	MX-2012-134460	22-05-2012	26-05-2012	Second Class	MC-18100	Mick Crebagga	Consumer	Tip

51290 rows × 24 columns

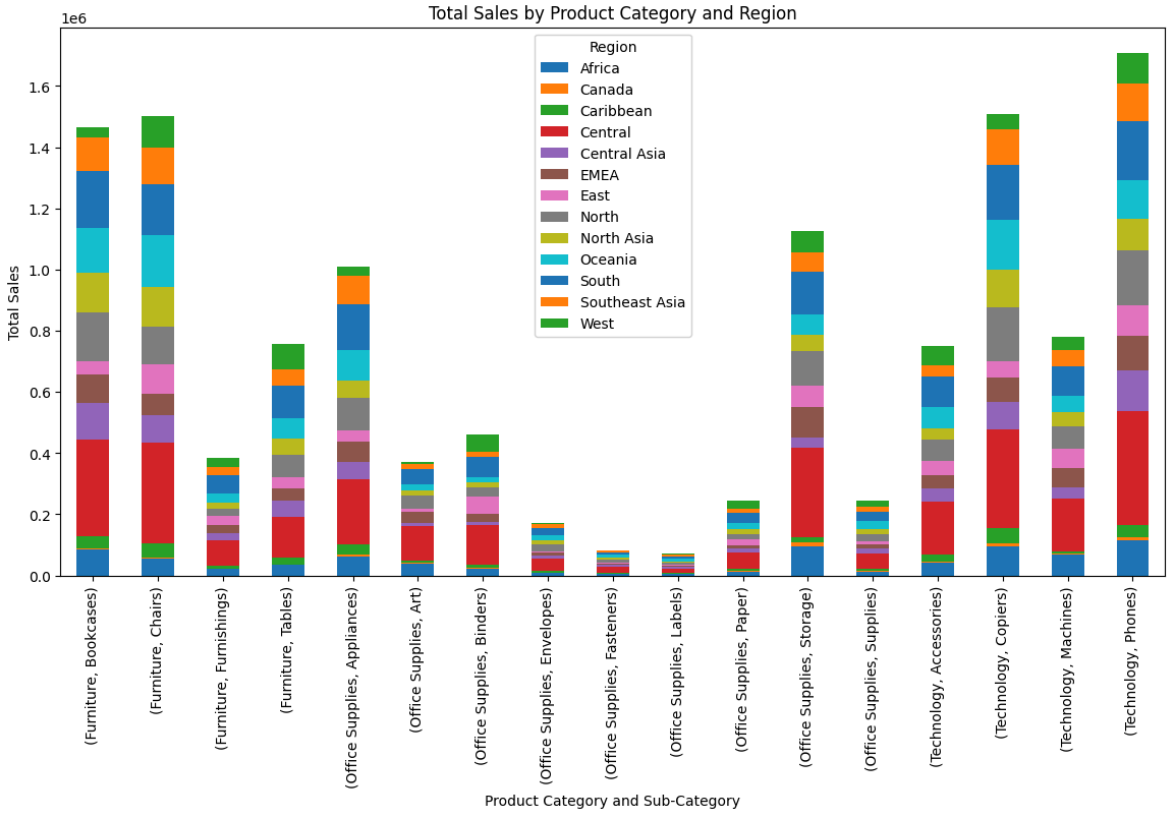
In [ ]:
```
# Group by Category, Sub-Category, and Region

category_region_sales = data.groupby(['Category', 'Sub-Category', 'Region'])['Sa
category_region_profit = data.groupby(['Category', 'Sub-Category', 'Region'])['P
```

In [ ]:
```
# Pivot tables for easier plotting
sales_pivot = category_region_sales.pivot_table(index=['Category', 'Sub-Category
profit_pivot = category_region_profit.pivot_table(index=['Category', 'Sub-Catego
```

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

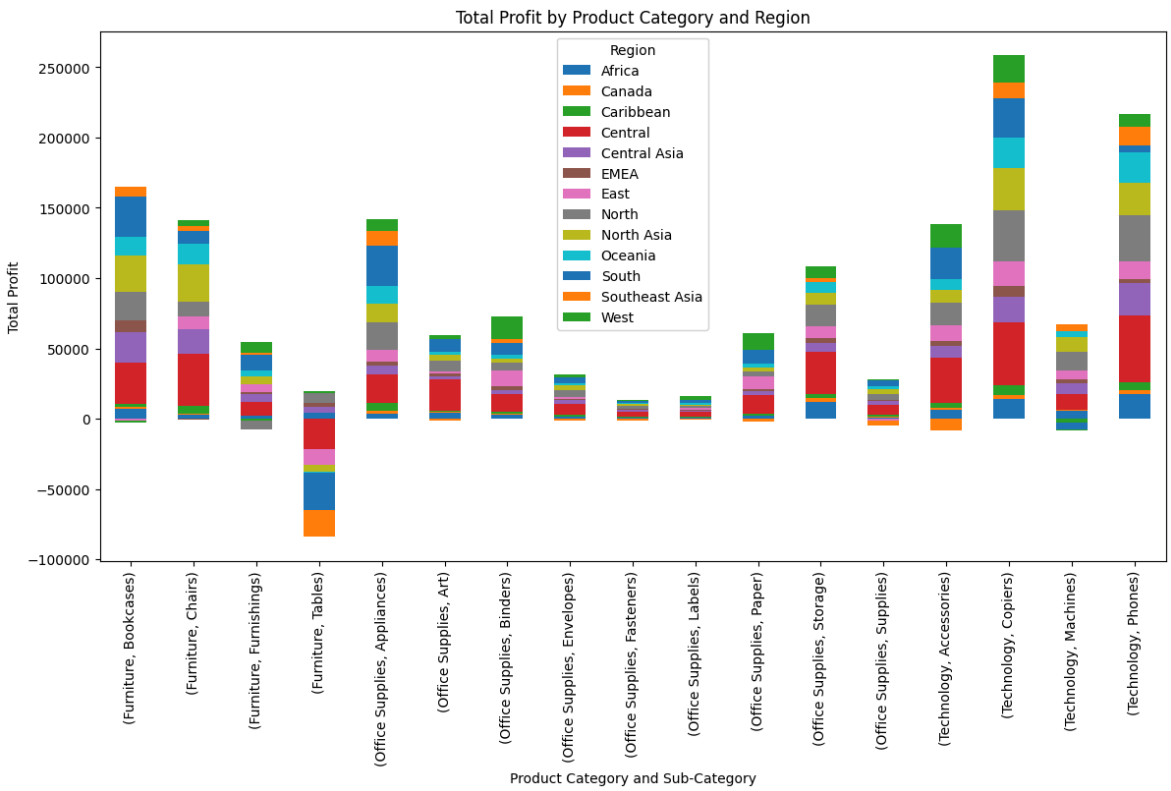
In [ ]:
```
# Plotting Sales by Region
plt.figure(figsize=(14, 7))
sales_pivot.plot(kind='bar', stacked=True, figsize=(14, 7))
plt.title('Total Sales by Product Category and Region')
plt.xlabel('Product Category and Sub-Category')
plt.ylabel('Total Sales')
plt.legend(title='Region')
plt.show()
```

```
<Figure size 1400x700 with 0 Axes>
```



In [ ]:
```
# Plotting Profit by Region
plt.figure(figsize=(14, 7))
profit_pivot.plot(kind='bar', stacked=True, figsize=(14, 7))
plt.title('Total Profit by Product Category and Region')
plt.xlabel('Product Category and Sub-Category')
plt.ylabel('Total Profit')
plt.legend(title='Region')
plt.show()
```

```
<Figure size 1400x700 with 0 Axes>
```



In [ ]:
```
sales_percentage = category_region_sales.copy()
sales_percentage = sales_percentage.sort_values(by='Sales', ascending = False)
sales_percentage
```

Out[ ]:

	Category	Sub-Category	Region	Sales
211	Technology	Phones	Central	370198.04400
16	Furniture	Chairs	Central	328571.43700
185	Technology	Copiers	Central	322961.87864
3	Furniture	Bookcases	Central	315419.41480
146	Office Supplies	Storage	Central	292402.30900
...	...	...	...	...
110	Office Supplies	Fasteners	East	819.71800
27	Furniture	Furnishings	Canada	808.53000
92	Office Supplies	Envelopes	Canada	567.18000
105	Office Supplies	Fasteners	Canada	479.46000
118	Office Supplies	Labels	Canada	421.74000

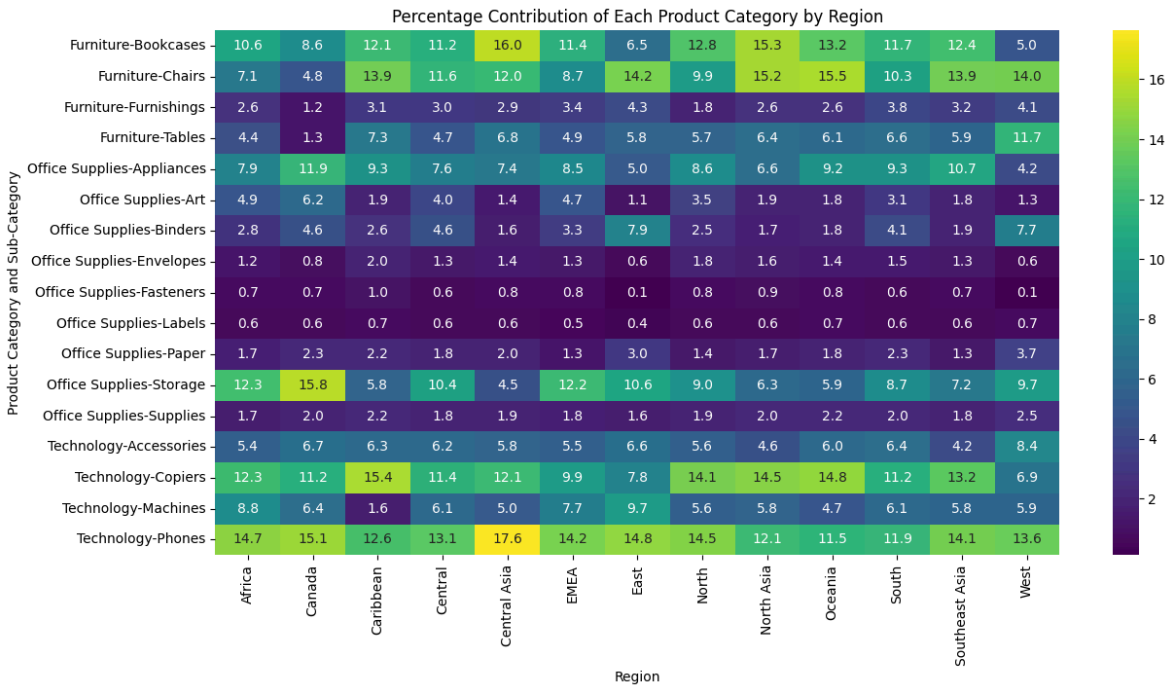
221 rows × 4 columns

In [ ]:
```
#Calculate percentage contribution based on region and sales
total_sales_by_region = sales_percentage.groupby('Region')['Sales'].sum().reset_
total_sales_by_region.columns = ['Region', 'Total Sales']
```

In [ ]:
```
# Merge total sales back into the main DataFrame
sales_percentage = sales_percentage.merge(total_sales_by_region, on='Region')
```

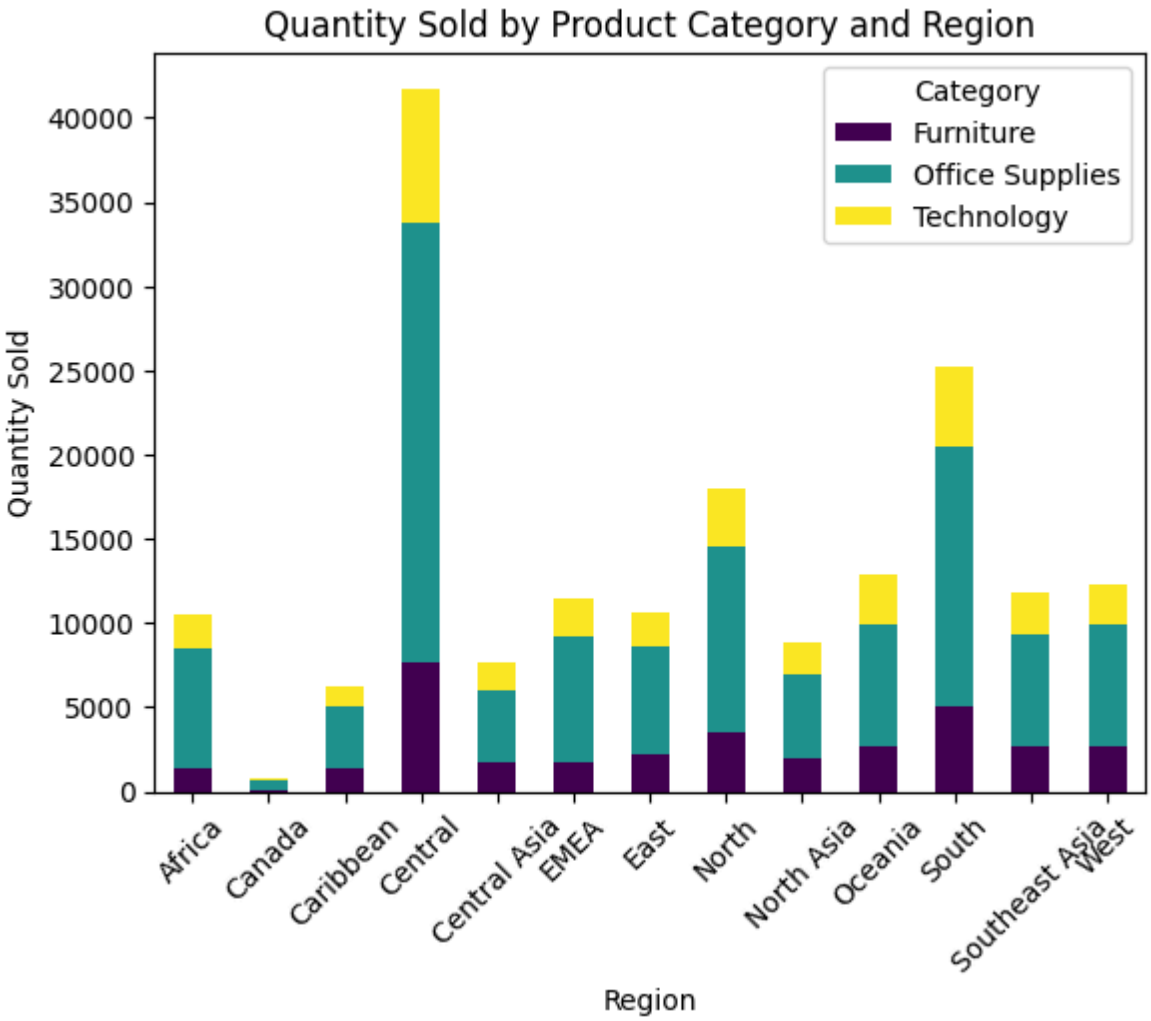
In [ ]:
```
# Calculate percentage contribution
sales_percentage['Percentage'] = (sales_percentage['Sales'] / sales_percentage['
sales_percentage_pivot = sales_percentage.pivot_table(index=['Category', 'Sub-Ca
```

In [ ]:
```
# Plotting Percentage Contribution
plt.figure(figsize=(14, 7))
sns.heatmap(sales_percentage_pivot, cmap='viridis', annot=True, fmt=".1f")
plt.title('Percentage Contribution of Each Product Category by Region')
plt.xlabel('Region')
plt.ylabel('Product Category and Sub-Category')
plt.show()
```



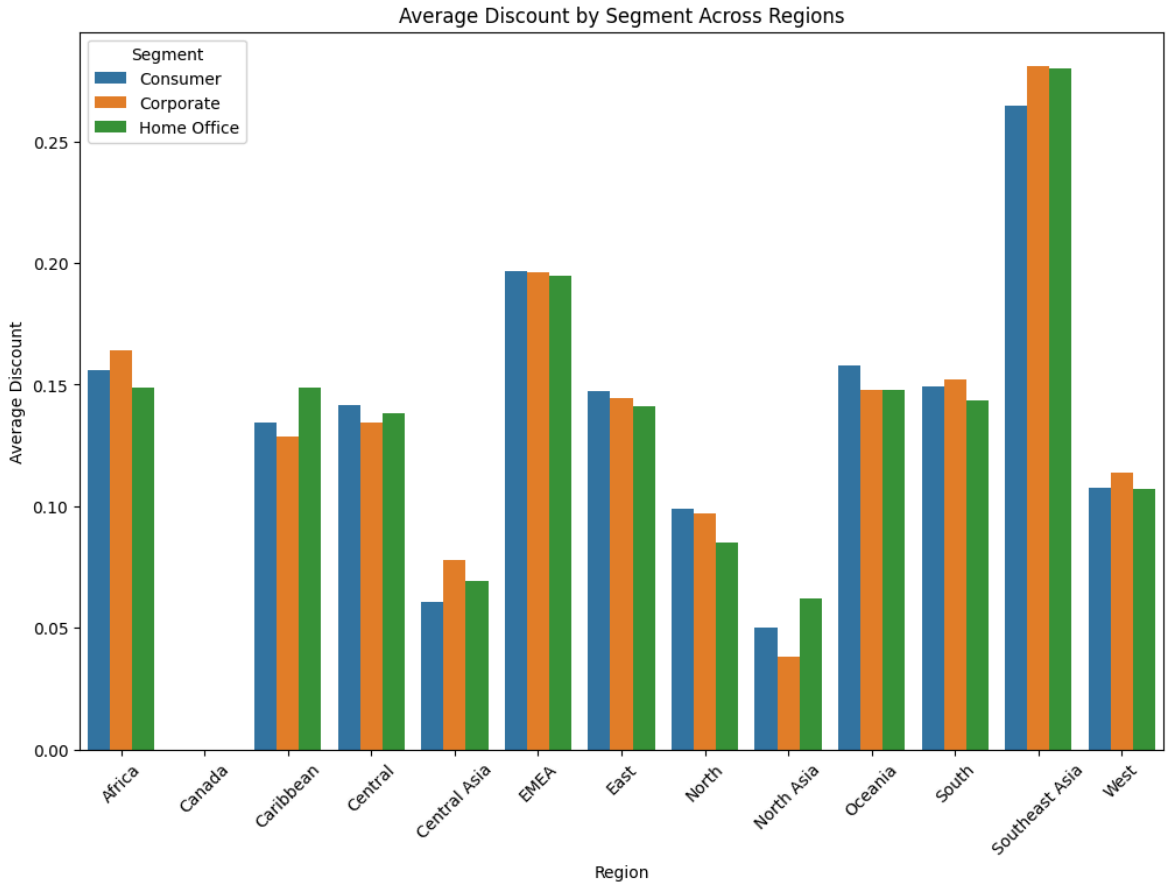
In [ ]:
```
#Quantity Sold by Product Category and Region
quantity_sold_by_region = data.groupby(['Region', 'Category'])['Quantity'].sum()
quantity_sold_by_region = quantity_sold_by_region.pivot(index='Region', columns=

# Plot Quantity Sold by Product Category and Region
plt.figure(figsize=(14, 7))
quantity_sold_by_region.plot(kind='bar', stacked=True, colormap='viridis')
plt.title('Quantity Sold by Product Category and Region')
plt.xlabel('Region')
plt.ylabel('Quantity Sold')
plt.xticks(rotation=45)
plt.legend(title='Category')
plt.show()
```

```
<Figure size 1400x700 with 0 Axes>
```



In [ ]:
```
# Remove duplicate columns
data = data.loc[:, ~data.columns.duplicated()]

# Clean column names
data.columns = data.columns.str.strip()

# Convert to numeric
data['Profit'] = pd.to_numeric(data['Profit'], errors='coerce')
```

In [ ]:
```
#Average Discount Given by Segment Across Regions:

avg_discount_by_segment_region = data.groupby(['Segment', 'Region'])['Discount']
avg_discount_by_segment_region.columns = ['Segment', 'Region', 'Average Discount

# Plotting the average discount by segment and region
plt.figure(figsize=(12, 8))
sns.barplot(x='Region', y='Average Discount', hue='Segment', data=avg_discount_b
plt.title('Average Discount by Segment Across Regions')
plt.xlabel('Region')
plt.ylabel('Average Discount')
plt.legend(title='Segment')
plt.xticks(rotation=45)
plt.show()
```



In [ ]:
```
avg_discount_by_segment_region
```

Out[ ]:

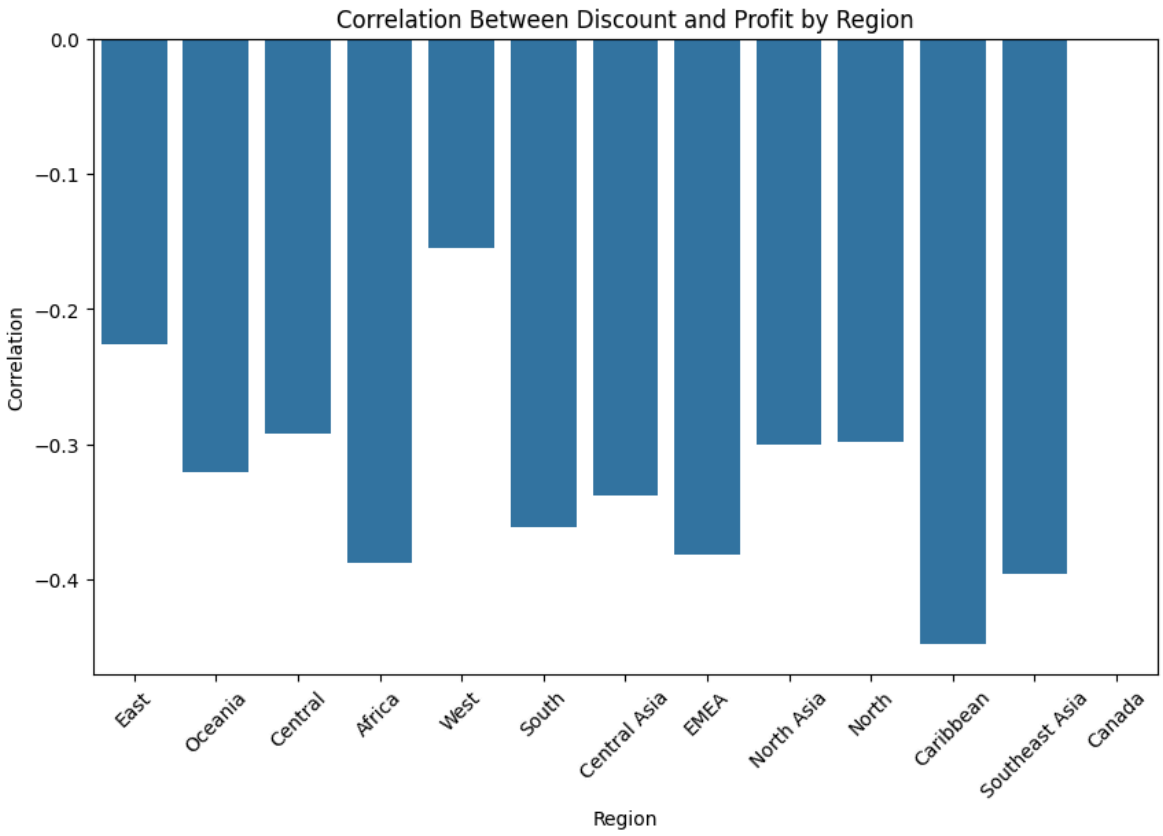
	Segment	Region	Average Discount
0	Consumer	Africa	0.155733
1	Consumer	Canada	0.000000
2	Consumer	Caribbean	0.134435
3	Consumer	Central	0.141548
4	Consumer	Central Asia	0.060749
5	Consumer	EMEA	0.196690
6	Consumer	East	0.147447
7	Consumer	North	0.099059
8	Consumer	North Asia	0.050000
9	Consumer	Oceania	0.157975
10	Consumer	South	0.149212
11	Consumer	Southeast Asia	0.264461
12	Consumer	West	0.107506
13	Corporate	Africa	0.163948
14	Corporate	Canada	0.000000
15	Corporate	Caribbean	0.128746
16	Corporate	Central	0.134564
17	Corporate	Central Asia	0.077651
18	Corporate	EMEA	0.195934
19	Corporate	East	0.144356
20	Corporate	North	0.097243
21	Corporate	North Asia	0.037994
22	Corporate	Oceania	0.147768
23	Corporate	South	0.151928
24	Corporate	Southeast Asia	0.280935
25	Corporate	West	0.113958
26	Home Office	Africa	0.148658
27	Home Office	Canada	0.000000
28	Home Office	Caribbean	0.148828
29	Home Office	Central	0.138178
30	Home Office	Central Asia	0.069466
31	Home Office	EMEA	0.194656
32	Home Office	East	0.141036

	Segment	Region	Average Discount
33	Home Office	North	0.085002
34	Home Office	North Asia	0.061957
35	Home Office	Oceania	0.147906
36	Home Office	South	0.143644
37	Home Office	Southeast Asia	0.280000
38	Home Office	West	0.106918

In [ ]:

```
# Create an empty dictionary to store the correlations
correlation_by_region = {}

# Loop through each region and calculate the correlation
for region in data['Region'].unique():
    # Filter the data for the current region
    region_data = data[data['Region'] == region]

    # Calculate the correlation between Discount and Profit
    correlation = region_data['Discount'].corr(region_data['Profit'])

    # Store the result in the dictionary
    correlation_by_region[region] = correlation

# Convert the dictionary to a DataFrame
correlation_by_region_df = pd.DataFrame(list(correlation_by_region.items()), col
correlation_by_region_df
plt.figure(figsize=(10, 6))
sns.barplot(x='Region', y='Discount-Profit Correlation', data=correlation_by_reg
plt.title('Correlation Between Discount and Profit by Region')
plt.xlabel('Region')
plt.ylabel('Correlation')
plt.xticks(rotation=45)
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2922: Ru
ntimeWarning: invalid value encountered in divide
  c /= stddev[:, None]
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2923: Ru
ntimeWarning: invalid value encountered in divide
  c /= stddev[None, :]
```



In [ ]:
```
data.head(3)
```

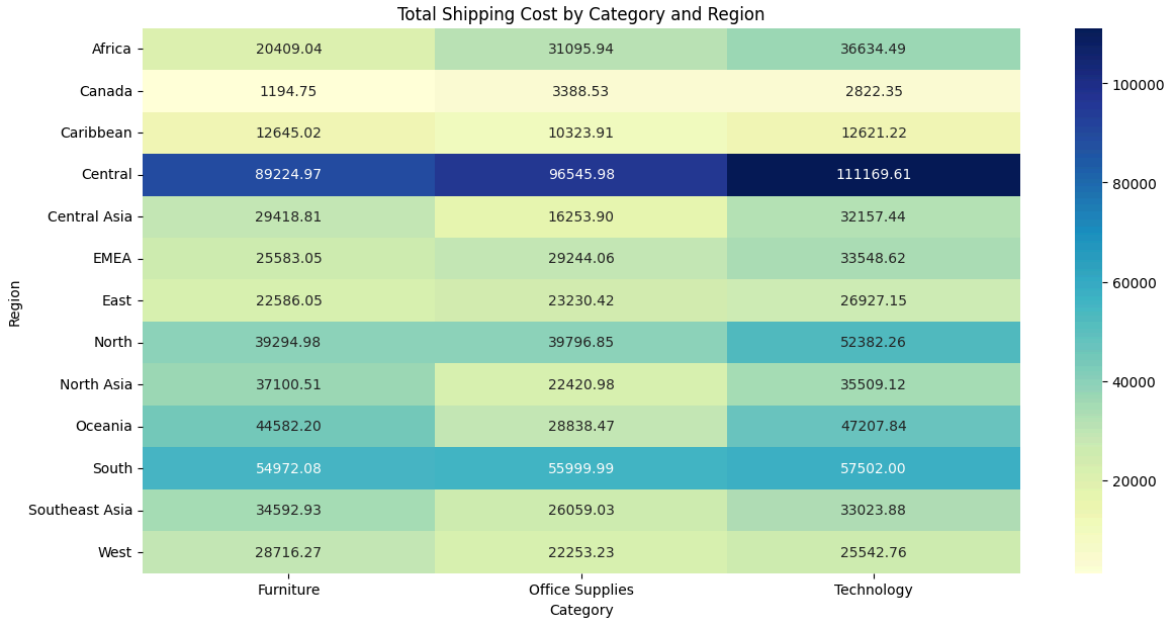
Out[ ]:

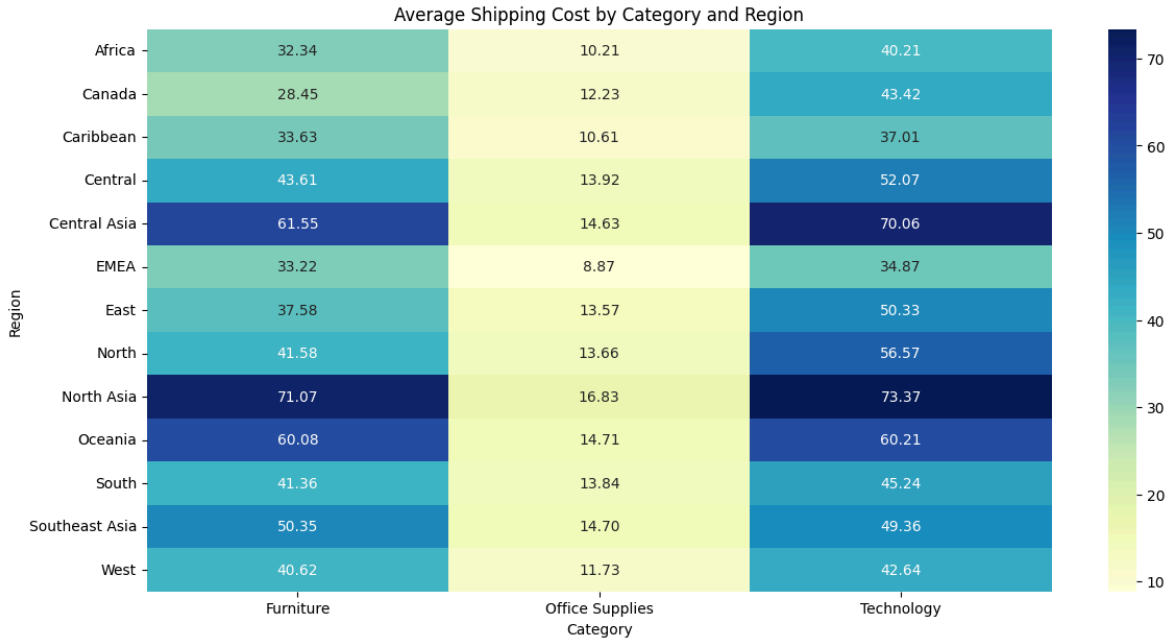
	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	31-07-2012	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	05-02-2013	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong
2	25330	IN-2013-71249	17-10-2013	18-10-2013	First Class	CR-12730	Craig Reiter	Consumer	Brisbane

3 rows × 24 columns

In [ ]:
```
#Examine the relationship between sales and shipping costs across regions.
# Total and Average Shipping Cost by Category and Region
shipping_by_cat_reg = data.groupby(['Category', 'Region'])['Shipping Cost'].agg(
shipping_by_cat_reg.columns = ['Category', 'Region', 'Total Shipping Cost', 'Ave

# Pivot tables for visualization
```

```
total_shipping_pivot = shipping_by_cat_reg.pivot(index='Region', columns='Catego
average_shipping_pivot = shipping_by_cat_reg.pivot(index='Region', columns='Cate

# Plot Total Shipping Costs
plt.figure(figsize=(14, 7))
sns.heatmap(total_shipping_pivot, annot=True, cmap='YlGnBu', fmt='.2f')
plt.title('Total Shipping Cost by Category and Region')
plt.xlabel('Category')
plt.ylabel('Region')
plt.show()

# Plot Average Shipping Costs
plt.figure(figsize=(14, 7))
sns.heatmap(average_shipping_pivot, annot=True, cmap='YlGnBu', fmt='.2f')
plt.title('Average Shipping Cost by Category and Region')
plt.xlabel('Category')
plt.ylabel('Region')
plt.show()
```





In [ ]:
```
shipping_by_cat_reg
```

Out[ ]:

	Category	Region	Total Shipping Cost	Average Shipping Cost
0	Furniture	Africa	20409.04	32.343962
1	Furniture	Canada	1194.75	28.446429
2	Furniture	Caribbean	12645.02	33.630372
3	Furniture	Central	89224.97	43.609467
4	Furniture	Central Asia	29418.81	61.545628
5	Furniture	EMEA	25583.05	33.224740
6	Furniture	East	22586.05	37.580782
7	Furniture	North	39294.98	41.581989
8	Furniture	North Asia	37100.51	71.073774
9	Furniture	Oceania	44582.20	60.083827
10	Furniture	South	54972.08	41.363491
11	Furniture	Southeast Asia	34592.93	50.353610
12	Furniture	West	28716.27	40.617072
13	Office Supplies	Africa	31095.94	10.212131
14	Office Supplies	Canada	3388.53	12.232960
15	Office Supplies	Caribbean	10323.91	10.610391
16	Office Supplies	Central	96545.98	13.919547
17	Office Supplies	Central Asia	16253.90	14.629973
18	Office Supplies	EMEA	29244.06	8.869900
19	Office Supplies	East	23230.42	13.569171
20	Office Supplies	North	39796.85	13.657121
21	Office Supplies	North Asia	22420.98	16.832568
22	Office Supplies	Oceania	28838.47	14.706002
23	Office Supplies	South	55999.99	13.844250
24	Office Supplies	Southeast Asia	26059.03	14.697704
25	Office Supplies	West	22253.23	11.730749
26	Technology	Africa	36634.49	40.213491
27	Technology	Canada	2822.35	43.420769
28	Technology	Caribbean	12621.22	37.012375
29	Technology	Central	111169.61	52.070075
30	Technology	Central Asia	32157.44	70.059782
31	Technology	EMEA	33548.62	34.873825
32	Technology	East	26927.15	50.331121

	Category	Region	Total Shipping Cost	Average Shipping Cost
33	Technology	North	52382.26	56.568315
34	Technology	North Asia	35509.12	73.365950
35	Technology	Oceania	47207.84	60.214082
36	Technology	South	57502.00	45.241542
37	Technology	Southeast Asia	33023.88	49.363049
38	Technology	West	25542.76	42.642337

In [ ]:
```
data.head(2)
```

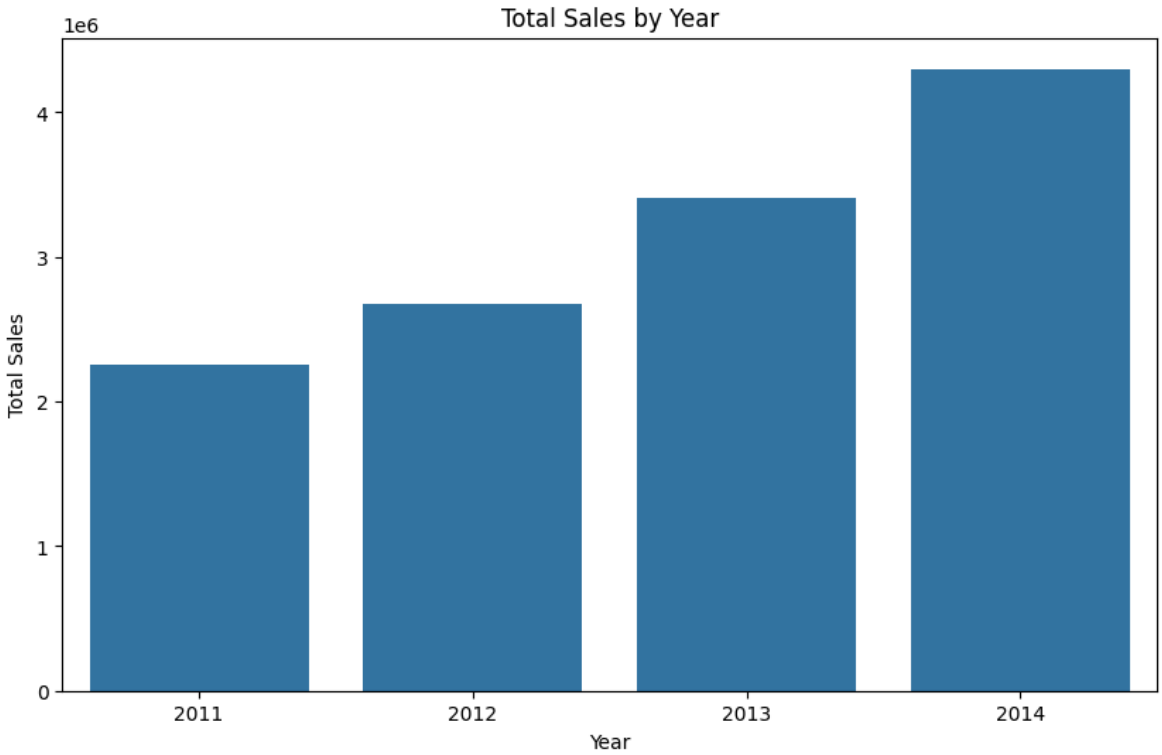
Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	31-07-2012	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	05-02-2013	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong

2 rows × 27 columns

In [ ]:
```
data['year'] = pd.to_datetime(data['Order Date']).dt.year
data['month'] = pd.to_datetime(data['Order Date']).dt.month
data['day'] = pd.to_datetime(data['Order Date']).dt.day


sales_year = data.groupby('year')['Sales'].sum().reset_index(name= 'Total Sales'
sales_year['YoY Growth (%)'] = sales_year['Total Sales'].pct_change() * 100

sales_year.columns = ['year','Total Sales','Y-o-Y']
sales_year

plt.figure(figsize=(10, 6))
sns.barplot(x='year', y='Total Sales', data=sales_year)
plt.title('Total Sales by Year')
plt.xlabel('Year')
plt.ylabel('Total Sales')
plt.show()
```

```
/tmp/ipykernel_4907/3268586548.py:1: UserWarning: Parsing dates in %d-%m-%Y forma
t when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specif
y a format to silence this warning.
  data['year'] = pd.to_datetime(data['Order Date']).dt.year
/tmp/ipykernel_4907/3268586548.py:2: UserWarning: Parsing dates in %d-%m-%Y forma
t when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specif
y a format to silence this warning.
  data['month'] = pd.to_datetime(data['Order Date']).dt.month
/tmp/ipykernel_4907/3268586548.py:3: UserWarning: Parsing dates in %d-%m-%Y forma
t when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specif
y a format to silence this warning.
  data['day'] = pd.to_datetime(data['Order Date']).dt.day
```



In [ ]:
```
sales_year
```

Out[ ]:

	year	Total Sales	Y-o-Y
0	2011	2.259451e+06	NaN
1	2012	2.677439e+06	18.499530
2	2013	3.405746e+06	27.201659
3	2014	4.299866e+06	26.253259